

Mater — PRD: Vehicle Detection v1 (Entry/Exit + Occupancy)

Sep 24, 2026 · @Valen

Overview

v1 adds camera-based vehicle detection to the existing Mater dashboard: it logs every car entering and exiting a lot, knows which cars are on the lot now, and tracks whether monitored spaces are occupied.

Mater dispatches tow trucks to private-property violations so drivers never patrol. Today drivers circle apartment lots and garages looking for violators, and many trips end with no tow because the car left or the tow could not be authorized. Mater only sends a truck when the tow is a sure thing.

This PRD covers the detection layer only. Dispatch, the driver app, permits and tow paperwork are later PRDs. v1's output feeds the existing `POST /api/ingest-scan-event` endpoint in the `mater-demo` repo.

Goals and non-goals

Goals

1. Detect and track vehicles in a video file, RTSP stream or YouTube live stream.
2. Classify each vehicle crossing a virtual line as an entry or an exit.
3. Read the plate when visible, keeping the most confident read per vehicle.
4. Classify each monitored parking space as occupied or empty, with change events.
5. Post every event to the existing ingest endpoint with a new `direction` field.
6. Show an "On the lot now" list and live space occupancy in the dashboard.
7. Ship an evaluation harness that scores accuracy against hand-labeled ground truth.

Non-goals for v1

- Tow dispatch, the driver app, truck GPS.
- Permit lists and violation rules (v2; v1 only stores plates).
- Tow paperwork, police or owner notices.
- Custom model training. v1 uses pretrained models only.

- Buying or installing camera hardware.

Existing system (mater-demo repo)

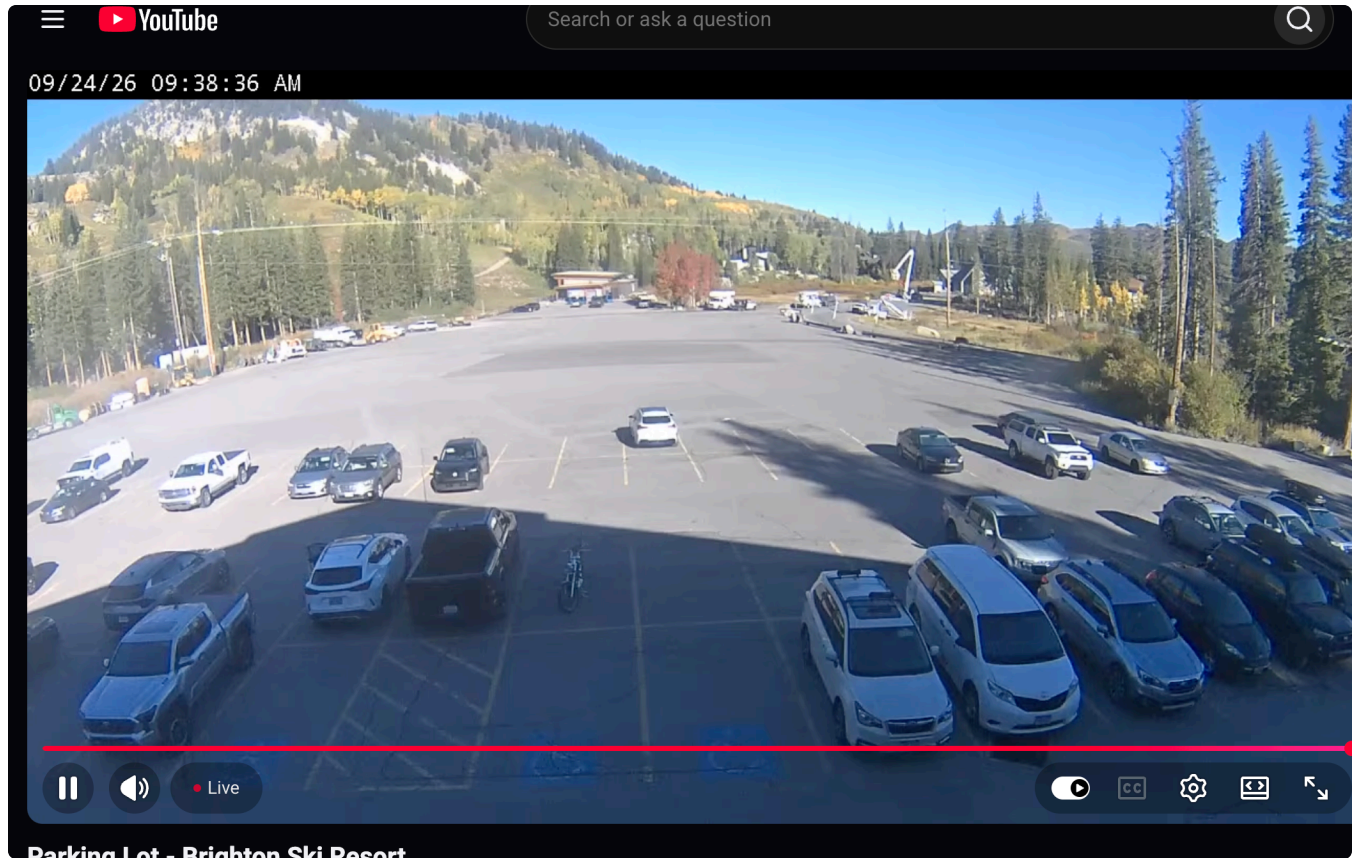
Most of the backend already exists; v1 extends it rather than rebuilding it. Repo: `github.com/AntoDono/mater-demo` (private). The dashboard lives in `dashboard/`, migrations in `supabase/migrations/`.

Area	What exists today	v1 change
Stack	Next.js 14 App Router, TypeScript, Supabase (Auth, Postgres, Storage, Realtime), Tailwind, Mapbox GL	None
Tenancy	<code>orgs</code> , <code>org_members</code> , RLS on every table	New tables get RLS by <code>org_id</code>
Regions	<code>regions</code> with GeoJSON polygons, time limits, auto-ticketing flag	Add cameras and spaces under a region
Scans	<code>vehicle_scans</code> with timers and status; <code>scan_events</code> evidence timeline (photos + timestamps)	Add <code>direction</code> , <code>camera_id</code> , <code>track_id</code> to events
Ingest	<code>POST /api/ingest-scan-event</code> : org, region, time, image URL, plate text + confidence, <code>external_vehicle_id</code> , source, metadata; 10-minute dedup window	Accept <code>direction</code> ; entry opens a presence, exit closes it
Expiry	<code>POST /api/process-expired</code> via cron; client polls every 20 s	Unchanged
Tickets / notifications	Manual and automatic tickets; realtime notifications	Rename "tickets" to "violations" in UI copy

Rule for Claude Code: read the existing schema, RLS policies and ingest route before writing anything, and extend them. Do not create a parallel ingest path.

Test video sources

The primary live test source is the Brighton Ski Resort parking lot livestream on YouTube: <https://www.youtube.com/watch?v=4a-3iEM7bHk>. It tests tracking, line crossing and occupancy, not plate reading.



What the Brighton view gives us, from the frame above:

- A fixed, elevated wide-angle camera over a marked lot, with about 25 vehicles visible in the fall off-season.
- Two near rows of clearly marked spaces in the foreground, good for occupancy ROIs.
- A central drive aisle where cars move, good for a virtual counting line.
- Hard cases: long shadows across the near rows, fisheye distortion at the edges, a motorcycle between cars, and a burned-in timestamp overlay (top left) to mask out.
- Plates are not readable at this distance. Do not score plate accuracy on this feed.
- Lot occupancy will swing hard once ski season starts, which makes it a good long-running test.

How to read it: resolve the live HLS URL with `yt-dlp -g <url>` and open it with OpenCV/ffmpeg. Re-resolve when the URL expires. Use it for internal testing only; do not redistribute or ship its footage.

Source	Use it to test	Notes
Brighton livestream	Tracking, entry/exit line logic, occupancy, long-running stability	Live; no plates
<code>supervision</code> sample videos (<code>supervision.assets</code>)	Line-crossing logic in the first hour	Traffic footage, downloaded by the library
Pexels / Pixabay clips ("parking garage entrance", "parking lot gate")	Entry/exit and plate reading at close range	Free license; check each clip
CNRPark-EXT, PKLot	Occupancy per space in varied weather and light	Research datasets; not US lots
UFPR-ALPR	Multi-frame plate reading per vehicle	Brazilian plates
Our own phone recordings	Real US entrances, night, rain	Written permission from the property owner first

Every recorded test clip gets a ground-truth CSV (see Evaluation) stored in `detector/eval/clips/`.

Functional requirements

The detector is a standalone Python worker that turns a video source into events; the dashboard stores and displays them.

FR1 — Video input

- Accept a local video file, an RTSP URL, or a YouTube URL (resolved via `yt-dlp`).
- Process at a configurable frame rate (default 5 fps) to control CPU use.
- Reconnect automatically on stream drop, with backoff; log each drop.
- Mask configurable regions (e.g. timestamp overlays) before detection.

FR2 — Detection and tracking

- Detect car, truck, bus and motorcycle with a pretrained YOLO model (Ultralytics).
- Track with ByteTrack so each vehicle keeps a stable `track_id` while in view.
- Drop detections below a configurable confidence (default 0.4).

FR3 — Entry and exit

- Each camera config defines one or more counting lines with an inward direction.
- A tracked vehicle crossing a line inward emits `entry` ; outward emits `exit` .
- Emit once per crossing; ignore jitter back and forth within 2 seconds.

FR4 — Plate reading

- Run plate detection + OCR (`fast-alpr`) on vehicle crops while the track is in view.
- Keep the highest-confidence read per track; attach it to the crossing event.
- If no read reaches the threshold (default 0.8), send the event with no plate.

FR5 — Space occupancy

- Each camera config can define parking spaces as image polygons with a space label.
- A space is occupied when a vehicle box overlaps it above a threshold (default 40%).
- Emit `occupied` / `vacated` only after the state holds for 30 seconds, to ignore passing cars.

FR6 — Evidence

- Save one JPEG snapshot per event (full frame, vehicle box drawn) to Supabase Storage.
- Store the snapshot URL on the event.

FR7 — Ingest

- Post crossing events to `/api/ingest-scan-event` with `direction` , `camera_id` , `track_id` (as `external_vehicle_id`), plate and confidence.
- Post occupancy changes to a new `/api/ingest-space-event` .
- Authenticate the worker with a per-camera API key, not the service-role key.
- Buffer events locally and retry if the API is unreachable.

FR8 — Dashboard

- **On the lot now:** vehicles with an entry and no matching exit, with plate, entry time and time on lot.
- **Entry/exit log:** every crossing with snapshot, filterable by region and camera.
- **Space map:** each monitored space shown occupied or empty, updating live via Supabase Realtime.
- **Camera health:** last frame time, fps, and stream drops per camera.

- **Camera setup:** paste a frame, draw counting lines and space polygons on it, save to the camera config.

Architecture, data model and API

One Python detector worker per camera feeds the existing Next.js API; Supabase stays the single source of truth.

```

flowchart LR
    V[Video source<br/>file / RTSP / YouTube] --> D[Detector worker<br/>Python]
    D -->|snapshots| S[(Supabase Storage)]
    D -->|crossings| I[/api/ingest-scan-event/]
    D -->|space changes| O[/api/ingest-space-event/]
    I --> P[(Postgres)]
    O --> P
    P -->|Realtime| UI[Dashboard<br/>Next.js]
```

The worker pulls its camera config (lines, spaces, masks) from the API at start and every 60 seconds.

Detector stack

- Python 3.11, `ultralytics` (YOLO, pretrained COCO weights, nano or small), `supervision` (ByteTrack, LineZone, PolygonZone), `fast-alpr`, `opencv-python`, `yt-dlp`, `httpx`.
- New top-level folder `detector/` with `pyproject.toml`, `detector/config.example.yaml`, a Dockerfile, and `detector/eval/`.
- Runs on a laptop CPU at 5 fps for one camera; GPU optional.

New migration `003_detection.sql`

Table	Key columns	Purpose
<code>cameras</code>	<code>id</code> , <code>org_id</code> , <code>region_id</code> , <code>name</code> , <code>source_url</code> , <code>config</code> (jsonb: lines, spaces, masks, thresholds), <code>api_key_hash</code> , <code>last_frame_at</code>	One row per camera
<code>spaces</code>	<code>id</code> , <code>org_id</code> , <code>region_id</code> , <code>camera_id</code> , <code>label</code> , <code>polygon</code> (jsonb, image coords), <code>is_reserved</code>	Monitored spaces

Table	Key columns	Purpose
<code>space_events</code>	<code>id</code> , <code>org_id</code> , <code>space_id</code> , <code>state</code> (occupied / vacated), <code>captured_at</code> , <code>image_url</code> , <code>track_id</code>	Occupancy history
<code>presences</code>	<code>id</code> , <code>org_id</code> , <code>region_id</code> , <code>plate_text</code> , <code>plate_confidence</code> , <code>entry_event_id</code> , <code>exit_event_id</code> , <code>entered_at</code> , <code>exited_at</code>	Open row = car on lot now

- `scan_events` gains `direction` (entry / exit / null), `camera_id`, `track_id`.
- RLS on every new table, scoped by `org_id`, matching `002_rls_policies.sql`.

API changes

- `POST /api/ingest-scan-event`: accept optional `direction`, `camera_id`, `track_id`. Entry opens a presence; exit closes the most recent open presence with the same plate in that region. An exit with no plate or no match is logged, not dropped.
- `POST /api/ingest-space-event` (new): `camera_id`, `space_id`, `state`, `captured_at`, `image_url`, optional `track_id`.
- `GET /api/cameras/:id/config` (new): returns the camera config for the worker.
- All three authenticate with the camera's API key in an `Authorization` header.

Evaluation and acceptance

v1 is done when the detector meets the targets below on at least three labeled clips, including one at night or in rain.

Harness

- `python -m detector.eval <clip> <truth.csv>` runs the pipeline offline and prints a score report.
- Ground truth CSV per clip: `time_s`, `event`, `direction_or_state`, `plate`, `space_label`, labeled by hand while watching the clip.
- Events match when they are within 3 seconds of each other and agree on type.
- Results are saved to `detector/eval/results/` so runs can be compared over time.

Metric	How it is scored	v1 target
Crossing recall	Labeled crossings the system caught	$\geq 95\%$

Metric	How it is scored	v1 target
Crossing precision	System crossings that were real	≥ 95%
Direction accuracy	Caught crossings with the right entry/exit	≥ 98%
Plate exact match	Plates read exactly right, close-range daylight clips only	≥ 85%
Occupancy accuracy	Space states correct, sampled every 60 s	≥ 95%
Presence match	Exits paired to the right entry	≥ 90%
Stream uptime	Brighton livestream processed for 24 h unattended	≥ 99%, auto-reconnects
Latency	Crossing to row in the dashboard	≤ 5 s

Plate accuracy is below what towing needs. That is expected for v1: no tow will ever dispatch on a plate read alone, and later versions add human confirmation below a confidence threshold.

Milestones and build order

Build in six milestones, each ending with something runnable; Claude Code should finish and verify one before starting the next.

#	Milestone	Done when
M1	Detector skeleton: file/YouTube input, YOLO + ByteTrack, annotated output video	Brighton stream plays with stable track IDs drawn on cars
M2	Counting lines + occupancy polygons from a YAML config; events printed as JSON	A <code>supervision</code> sample clip and Brighton produce sensible entry/exit and occupancy events
M3	Eval harness + first ground-truth clips	Score report runs on 3 clips; baseline numbers saved
M4	Migration 003, ingest changes, camera API keys, snapshot upload	Events from Brighton land in Supabase and open/close presences

#	Milestone	Done when
M5	Dashboard: On the lot now, entry/exit log, space map, camera health, camera setup	A viewer sees Brighton update live with no page refresh
M6	Plate reading on close-range clips	Plate target met on daylight Pexels or own clips

Instructions for Claude Code

- ☐ Read `dashboard/README.md`, both existing migrations, and the ingest route before writing code.
- ☐ Keep the detector in `detector/`; do not add Python to the Next.js app.
- ☐ Extend existing tables and routes; never duplicate the ingest path.
- ☐ Every new table gets RLS by `org_id` in the same migration.
- ☐ Put every threshold in the camera config, never hard-coded.
- ☐ Add tests for line-crossing direction and presence matching (entry, exit, no-plate exit, duplicate exit).
- ☐ Never commit video, snapshots, API keys or `.env` files; add them to `.gitignore`.
- ☐ After each milestone, run the eval harness and paste the score report into the PR description.

Risks, privacy and open questions

The biggest technical risk is plate reading at night; the biggest business risk is mishandling plate data.

Risks

- **Night plates:** phone and webcam footage blurs plates in low light. Mitigation: score daylight separately; budget for an IR LPR camera after v1.
- **Missed exits:** a car leaving unseen leaves a stale presence. Mitigation: flag presences older than a configurable limit for review, never for a tow.
- **Occlusion and shadows:** as on Brighton's near rows. Mitigation: 30-second hold on state changes; tune overlap per camera.
- **YouTube stream URLs expire:** Mitigation: re-resolve with `yt-dlp` on every reconnect.

- **CPU load with several cameras:** Mitigation: frame skipping and the nano model by default.

Privacy

- Record or connect to a private camera only with the property owner's written permission.
- Store plates only for regions with an active agreement; delete presences and snapshots after a configurable retention period (default 30 days).
- Snapshots stay in private Supabase Storage, served through signed URLs.
- Never use unsecured private camera feeds found online.

Open questions

- ☐ Which property will give us real entrance footage first?
- ☐ What retention period will property managers accept?
- ☐ Do we match exits by plate only, or also by vehicle appearance when no plate is read?
- ☐ Which IR camera model do we buy for the first pilot?